

Decoupled Static Indexing and Parameterized Search for Approximate MIPS over Learned Sparse Representations

Roger Canal Estrada

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
`roger.canal@estudiantat.upc.edu`

Abstract. Learned Sparse Representations (LSR), such as SPLADE, preserve lexical interpretability but induce an extreme Maximum Inner Product Search (MIPS) latency bottleneck in high-dimensional sparse spaces. This paper presents a decoupled architecture with two explicit phases: a high-capacity static index partitioned by document clustering, and a parameterized online search engine based on Coordinate-at-a-Time (CaaT) traversal. The static phase preserves the candidate pool with large fixed index capacity, while the dynamic phase controls latency–recall trade-offs through runtime constraints. On the Natural Questions benchmark (2.68M documents, 30,522 dimensions), the system reaches the target $\text{Recall@30} \geq 0.90$ at approximately 40 ms average latency and exposes direct search-time tuning levers that further reduce latency without index rebuilding.

Keywords: Learned Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Inverted Indexing

1 Introduction

LSR models map semantics into vocabulary space, so retrieval remains sparse and interpretable [2]. However, LSR query weights do not follow the same heavy-head Zipfian pattern that classical lexical pipelines exploit. Instead, many coordinates carry moderate but non-negligible mass (often described as *wacky weights*) [4]. This geometry weakens the pruning selectivity of traditional WAND-like and Document-at-a-Time (DaaT) traversal because partial-score upper bounds become less decisive at early stages [1].

This work addresses this limitation with block-level reasoning. The key idea is to decompose each posting list into geometrically coherent blocks and score blocks through tight upper bounds before touching documents. This strategy shifts computation from expensive sparse dot products to inexpensive block-level comparisons.

The paper contributes the following elements. First, it defines a decoupled architecture where a high-capacity static index preserves coverage and a parameterized dynamic search loop controls runtime cost. Second, it enforces exact-maximum summary bounds at index construction, which makes block upper

bounds mathematically safe for aggressive skipping. Third, it demonstrates that this separation yields a stable operating point above Recall@30 = 0.90 while preserving independent search-time control of latency.

2 System Architecture

The system is built around a custom partitioned inverted index designed to support efficient approximate Maximum Inner Product Search over sparse vectors. The central goal of this data structure is to organize document information so that the search engine can quickly identify the most similar documents to any incoming query without scoring every document individually.

The full pipeline is organized into two phases. The *Static Index Phase* (Section 2.1) performs the **InvertedBlock** construction, and the *Parameterized Search Phase* (Section 2.2) computes the query search.

2.1 Static Index Phase

Document Clustering Step. The method partitions the document set $\mathcal{D} = \{d_i\}_{i=1}^N$ into K clusters by a sparse K-means objective:

$$\max_{\{\mathcal{C}_k\}, \{\mu_k\}} \sum_{k=1}^K \sum_{d_i \in \mathcal{C}_k} \langle d_i, \mu_k \rangle, \quad (1)$$

with assignment step

$$a(i) = \arg \max_{k \in \{1, \dots, K\}} \langle d_i, \mu_k \rangle, \quad (2)$$

and centroid update

$$\mu_k \leftarrow \frac{\sum_{d_i \in \mathcal{C}_k} d_i}{\left\| \sum_{d_i \in \mathcal{C}_k} d_i \right\|_2}. \quad (3)$$

Both operations are computed in sparse form. This is a *hard clustering*: each document d_i is assigned to exactly one cluster $\mathcal{C}_{a(i)}$, and no document belongs to more than one cluster.

Partitioned Inverted Index Construction. The index maps each concept t to a list of **InvertedBlocks**. Each block stores one cluster id and truncated doc-weight pairs. For each cluster c , the method also builds a dense summary vector $S_c \in \mathbb{R}^{|V|}$ with

$$S_c[t] = \max_{d \in \mathcal{C}_c} d[t]. \quad (4)$$

This is the absolute maximum weight of concept t inside cluster c .

For each concept t , the index keeps at most nb cluster blocks and at most nd documents per retained block. Exact maxima make the cluster upper bound mathematically valid (never under-estimated).

$$UB_c(q) = \sum_{t \in \text{supp}(q)} q[t] \cdot S_c[t] \quad (5)$$

Algorithm 1 Static Index Construction with Exact-Maximum Summaries

Require: Sparse documents, cluster assignments, nb , nd
Ensure: Partitioned index I and summary vectors S

- 1: **for** each cluster c **do**
- 2: initialize dense summary $S_c \leftarrow 0$
- 3: **for** each document $d \in \mathcal{C}_c$ **do**
- 4: **for** each non-zero (t, w) in d **do**
- 5: $S_c[t] \leftarrow \max(S_c[t], w)$
- 6: **end for**
- 7: **end for**
- 8: **end for**
- 9: **for** each concept t **do**
- 10: aggregate candidate docs by cluster and concept weight
- 11: keep top- nb clusters by accumulated weight on t
- 12: **for** each retained cluster c **do**
- 13: keep top- nd docs by weight on t
- 14: append `InvertedBlock`($c, doc_ids, weights$) to $I[t]$
- 15: **end for**
- 16: **end for**
- 17: **return** I, S

This property prevents miss-skipping. If summaries under-estimate, the system can discard clusters that contain true top- k documents.

2.2 Search Phase

Coordinate-at-a-Time (CaaT) Traversal The search engine processes incoming sparse queries over the static index with CaaT traversal. It parses non-zero query coordinates, sorts them by descending weight, and enters the inverted lists of each coordinate in turn.¹

Algorithm 2 formalizes the CaaT loop.

Aggressive Dynamic Pruning Heuristics The pruning stage rejects blocks by the inequality

$$UB_c(q) < H.\min() \cdot \text{heap_factor}. \quad (6)$$

If the inequality holds, the block cannot improve the current top- k under the selected confidence multiplier.

This rule replaces expensive document-level scoring with $O(1)$ float comparisons at block granularity. The search engine exposes four hyperparameters that restrict traversal without changing the static index:

¹ CaaT is preferred over classical Document-at-a-Time (DaaT) traversal [1] because it visits the highest-signal query coordinates first, rapidly inflating the min-heap threshold. A high threshold amplifies subsequent block-skipping decisions. DaaT, by contrast, mixes low- and high-signal evidence across all documents before the threshold stabilizes, reducing early pruning power.

Algorithm 2 Parameterized CaaT Search

Require: Query q , index I , summaries S , top- k , max_query_terms ,
 max_search_blocks , $heap_factor$, md

Ensure: Top- k result heap H

- 1: $Q \leftarrow$ non-zero query coordinates sorted by weight descending
- 2: **if** $max_query_terms > 0$ **then**
- 3: truncate Q to first max_query_terms terms
- 4: **end if**
- 5: compute $UB_c(q)$ for all clusters c
- 6: initialize min-heap H , visited set, $docs_examined \leftarrow 0$
- 7: **for** each term $t \in Q$ **do**
- 8: $blocks_seen \leftarrow 0$
- 9: **for** each block $b \in I[t]$ **do**
- 10: **if** $max_search_blocks > 0$ and $blocks_seen \geq max_search_blocks$ **then**
- 11: **break**
- 12: **end if**
- 13: $blocks_seen \leftarrow blocks_seen + 1$
- 14: **if** $UB_{b.cluster.id}(q) < H.min() \cdot heap_factor$ **then**
- 15: **continue**
- 16: **end if**
- 17: **for** each doc d in b **do**
- 18: **if** d not visited **then**
- 19: score $\leftarrow \langle q, d \rangle$ and update H
- 20: mark visited; $docs_examined \leftarrow docs_examined + 1$
- 21: **if** $md > 0$ and $docs_examined \geq md$ **then**
- 22: **return** H
- 23: **end if**
- 24: **end if**
- 25: **end for**
- 26: **end for**
- 27: **end for**
- 28: **return** H

1. `heap_factor`: relaxation multiplier controlling skip confidence.
2. `max_query_terms`: truncates low-signal query tail coordinates.
3. `max_search_blocks`: limits per-coordinate block breadth online.
4. `max_docs_to_visit` (md): hard computational budget bounding worst-case latency.

3 Experimental Evaluation

3.1 Experimental Setup

The evaluation uses the SISAP 2026 Task 3 dataset: Natural Questions (NQ) with 2.68M documents encoded as SPLADE sparse vectors in 30,522 dimensions. Recall@30 is measured against the provided exact k -NN ground truth,

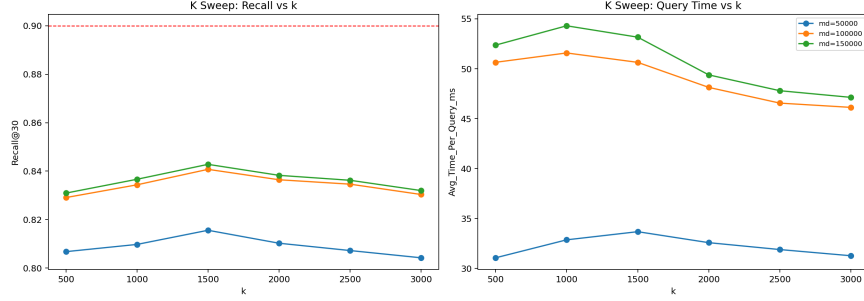


Fig. 1. Recall@30 and average query time as a function of K at three document budgets. $K = 1500$ consistently maximizes recall; larger K reduces latency at the cost of marginal recall loss.

and average query latency is reported. This protocol aligns with established LSR evaluation practice [3].

Experimental methodology. All experiments follow a staged wide-to-narrow sweep. First, the cluster count K is swept over a wide range with fixed nb , nd , and md . The best K found is then fixed while nb and nd are swept jointly. Finally, with the index frozen, the search parameters md , `heap_factor`, `max_query_terms` (`mqt`), and `max_search_blocks` (`msb`) are swept individually. Each stage uses the optimal values from the preceding stage as fixed constants, and a narrow refinement sweep is run around the best point.

Hardware. All runs execute inside a Docker container constrained to 8 virtual CPUs and 24 GB RAM with swap disabled. The host machine is an AMD Ryzen 5 5600X (6 cores / 12 logical processors, base 3.70 GHz, 32 MB L3 cache).

3.2 Static Index Capacity and the Recall Ceiling

The static index capacity determines the maximum recall achievable by any search strategy: any document excluded at build time (Alg. 1) is permanently unrecoverable. The following two sections characterize how K , nb , and nd affect this ceiling.

Effect of Cluster Count K . A wide sweep over $K \in \{500, 1000, 1500, 2000, 2500, 3000\}$ is run with fixed $nb = 500$, $nd = 500$ at three document budgets $md \in \{50000, 100000, 150000\}$. Figure 1 summarize the results.

$K = 1500$ yields the highest recall at every budget (0.816 at $md = 50000$, 0.841 at $md = 100000$, 0.843 at $md = 150000$). Smaller K reduces latency because cluster centroids are coarser and each concept’s block list is shorter, but the recall ceiling drops. Larger K also reduces latency (fewer documents per cluster) while lowering recall because finer partitions produce smaller blocks and weaker candidates survive the nb/nd truncation. Based on this, $K = 1500$ is selected as the index constant for subsequent sweeps.

Table 1. Fastest configuration with Recall@30 ≥ 0.90 ($K = 1500$).

md	nb	nd	Recall@30	ms/q	Index time (s)
100000	1000	500	0.9219	54.7	273
100000	500	500	0.9222	54.8	218
100000	500	150	0.9395	57.6	235
100000	1000	150	0.9404	57.6	287
150000	500	500	0.9576	82.4	218
150000	1000	150	0.9592	86.8	287

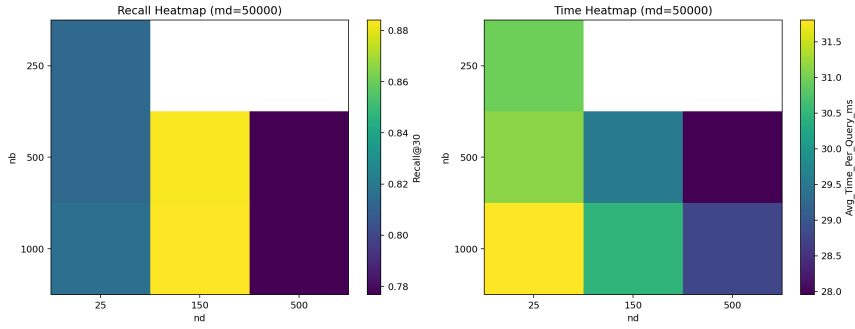


Fig. 2. Recall@30 heatmaps over nb and nd at $md = 50000$. Both axes raise the recall ceiling; the effect of nb is stronger at moderate md because more block entries survive the hard cap.

Effect of InvertedBlock parameters (nb) and (nd). With $K = 1500$ fixed, a joint sweep over $nb \in \{100, 500, 1000\}$ and $nd \in \{50, 150, 500\}$ is run at $md \in \{50000, 100000, 150000\}$. Figure 2 shows the recall heatmaps at $md = 50000$; Table 1 reports the fastest configuration exceeding Recall@30 = 0.90.

Both parameters raise the recall ceiling proportionally to index size. $nb = 500$, $nd = 500$ is selected as the operating default: it achieves Recall@30 = 0.922 at $md = 100000$ (54.8ms) with the lowest build time among configurations that exceed the task target, at 218s. Doubling nb to 1000 adds only ~ 0.002 recall while increasing build time by 24%.

3.3 Search Parameterization and Latency Optimization

With the static index fixed at $K = 1500$, $nb = 500$, $nd = 500$, the search parameters are swept independently. Algorithm 2 governs all four parameters.

Document Budget (md). md is the strongest recall lever. Table 2 and Figure 3 show its effect across $md \in \{25000, 30000, 50000, 75000, 100000, 125000, 150000\}$.

Recall and latency grow monotonically with md because the hard document cap is the binding stopping condition in every run. Average documents examined equals md in all cases, confirming that no block-skipping triggers before

Table 2. Effect of md on recall and latency ($K = 1500$, $nb = 500$, $nd = 500$, $hf = 0.05$).

md	Recall@30	ms/query	Avg. blocks entered
25000	0.682	14.1	238
30000	0.708	15.1	269
50000	0.775	25.7	382
75000	0.862	38.2	532
100000	0.921	51.5	698
125000	0.944	64.0	857
150000	0.957	77.4	1016

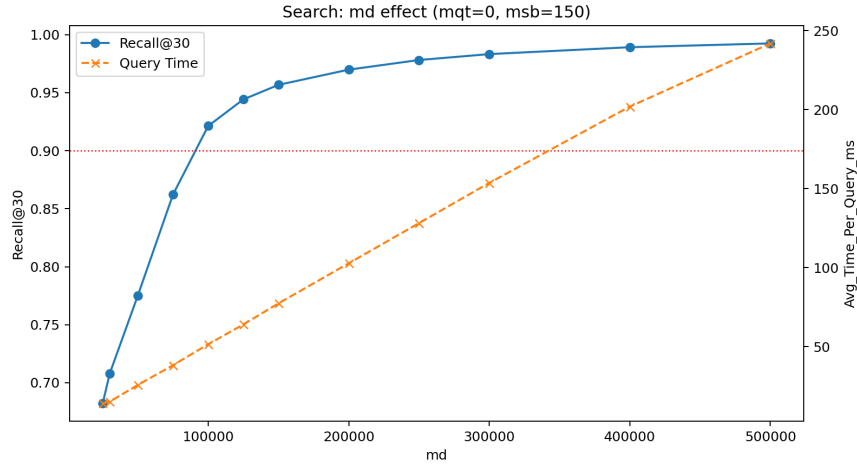


Fig. 3. Effect of md on Recall@30 and query latency. Both metrics grow monotonically. Recall@30 ≥ 0.90 is first met at $md = 100000$ (0.921 at 51.5 ms/q).

the budget is exhausted. md alone drives the recall–latency trade-off at these operating points.

Max Query Terms (mqt). mqt limits the number of highest-weighted query coordinates processed (0 = all). The sweep over $mqt \in \{0, 10, 20, 30, 40, 50, 100, 150, 200\}$ at $md = 50000$ and $md = 100000$ shows that recall and latency remain within ± 0.001 across the entire range (Figure 4). For SPLADE queries, md is exhausted before low-weight coordinates contribute additional candidates, so truncating the query tail has no measurable effect under the current operating regime.

Max Search Blocks (msb). msb limits the number of `InvertedBlocks` evaluated per query coordinate. Unlike mqt, msb has a strong effect at low values (Figure 5). At $md = 50000$, $msb = 10$ drops recall to 0.585; $msb = 40$ partially recovers to 0.766; $msb \geq 150$ saturates and matches the unlimited baseline (0.770).

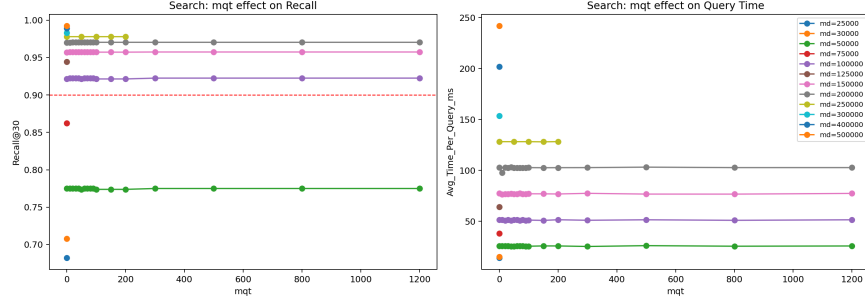


Fig. 4. Effect of mqt on Recall@30 and latency. Both metrics are flat across the full mqt range because *md* binds first.

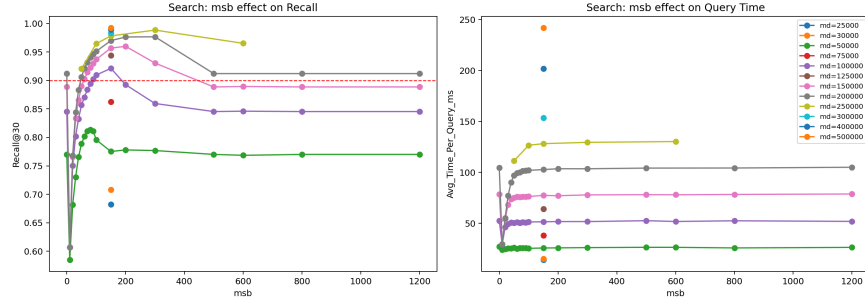


Fig. 5. Effect of msb on Recall@30. Recall collapses for $\text{msb} \leq 40$ and saturates at $\text{msb} \geq 150$, matching the unlimited baseline.

The relevant candidate clusters for each concept span at least 150 blocks, so any msb below that threshold causes systematic miss-skipping and recall collapse.

Heap Factor. Across all tested configurations, `heap_factor` has no measurable effect on recall or latency. Because *md* is the binding stopping condition in every run, the pruning inequality $UB_c(q) < H.\text{min}() \cdot \text{heap_factor}$ is almost never triggered before the budget is exhausted. `heap_factor` therefore acts as a theoretical safety valve rather than a practical performance lever under the current operating regime.

4 Conclusion and Future Work

This paper presents a mathematically grounded decoupled architecture that combines exact-maximum cluster summaries with CaaT traversal and block-level dynamic pruning. The static phase preserves a high-recall candidate pool, and the parameterized search phase provides direct runtime control of latency.

Future work targets two directions. First, automated search-parameter selection from query density statistics can adapt constraints per query. Second, memory-aligned forward-index layouts can accelerate sequential access during exact sparse dot products.

References

1. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., Zien, J.: Efficient query evaluation using a two-level retrieval process. In: *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)* (2003)
2. Formal, T., Piwowarski, B., Clinchant, S.: SPLADE: Sparse lexical and expansion model for first stage ranking. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021)
3. Thakur, N., Reimers, N., Rückle, A., Srivastava, A., Gurevych, I.: BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: *NeurIPS Datasets and Benchmarks Track* (2021)
4. Mackenzie, J., Trotman, A., Lin, J.: Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation. *arXiv preprint arXiv:2110.11540* (2021)
5. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024)
6. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Pairing clustered inverted indexes with κ -NN graphs for fast approximate retrieval over learned sparse representations. In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)* (2024)